# Reviewer Pack — Additive Energy Threshold Excludes ACC⁰ Circuits for Sipser …

Entry f5fad29c0f85 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 05:29:35 UTC

## Additive Energy Threshold Excludes ACC⁰ Circuits for Sipser Functions

**Entry ID**: `f5fad29c0f85` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-20 05:29:35 UTC

### 1. Conjecture

**Field A** (mathematical branch): Additive combinatorics **Field B** (complexity object): ACC⁰ circuit size

**Statement**:

> For every Sipser function S_n on n variables, its truth table's additive energy E(S_n) satisfies E(S_n) ≥ n^{2.5} - o(n^{2.5}), and any ACC⁰ circuit computing S_n has size ≥ Ω(n^{2.5}/log n).

**Rationale (proposer's reasoning)**:

> High additive energy implies structured correlations that resist ACC⁰ approximation, as additive energy quantifies the number of additive quadruples in the truth table, which are inherently non-linear and incompatible with low-depth threshold circuits.

**Taxonomy category**: `LIFTING` (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `7ef3e2f587077165`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> default: >=80% seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| RELATIVIZATION | UNCERTAIN | 0.95 | HITS | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.95 | UNCERTAIN | SAFE |
| ALGEBRIZATION | SAFE | 0.95 | UNCERTAIN | SAFE |

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| KARP_LIPTON | SAFE | 0.95 | SAFE | SAFE |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
from fractions import Fraction
from itertools import combinations

def sipser_function(n, x):
    return sum(x[i-1] for i in range(1, n+1)) % 2 == 0

def truth_table(f, n):
    return [[f(i) for i in range(2**n)]]

def additivity_quadruples(truth_table):
    n = len(truth_table[0])
    count = 0
    for a, b, c, d in combinations(range(n), 4):
        if truth_table[0][a] + truth_table[0][b] == truth_table[0][c] + truth_table[0][d]:
            count += 1
    return count

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    instances_tested = 30
    total_energy = 0
    support_count = 0

    for _ in range(instances_tested):
        x = tuple(random.randint(0, 1) for _ in range(n))
        f_value = sipser_function(n, x)
        truth_table_values = truth_table(f_value, n)
        energy = additivity_quadruples(truth_table_values)
        total_energy += energy

    mean_energy = Fraction(total_energy, instances_tested)
    conjecture_holds = mean_energy >= n**2.5 - 1e-6
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "additive_energy",
        "metric_value": float(mean_energy),
        "instances_tested": instances_tested,
```

```
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_energy = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_energy} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_energy} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_80cad486.py", line 63, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_80cad486.py", line 41, in run_trial
    truth_table_values = truth_table(f_value, n)
                         ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_80cad486.py", line 21, in truth_table
    return [[f(i) for i in range(2**n)]]
             ^^^^
TypeError: 'bool' object is not callable
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> Test crashed due to TypeError, preventing data collection. Support condition cannot be evaluated without successful trials. | next: Fix the truth_table function's boolean callable error and rerun tests

## 11. Audit log (LLM calls)

**Total LLM calls**: 12

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | qwen3:8b | 0 | 0 | 136444 |
| 2 | propose | ollama_remote | qwen3:8b | 0 | 0 | 107810 |
| 3 | propose | ollama_remote | qwen3:8b | 0 | 0 | 138576 |
| 4 | propose | ollama_remote | qwen3:8b | 0 | 0 | 101151 |
| 5 | preregistration | ollama_remote | qwen3:8b | 0 | 0 | 31739 |
| 6 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 28008 |
| 7 | novelty | ollama_remote | qwen3:8b | 0 | 0 | 19718 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 14721 |
| 9 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 7677 |
| 10 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 8847 |
| 11 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 7625 |
| 12 | judge | ollama_remote | qwen3:8b | 0 | 0 | 22365 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 624682 ms total latency. Provider mix: {'ollama_remote': 12}

*(full prompt+response transcripts available in `research/audit/f5fad29c0f85.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f5fad29c0f85.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/f5fad29c0f85.tar.gz` (if generated)